Lab Midterm – Web Technologies (CSC336)

Name: Muhammad Hammad Sarwar **Registration No.:** FA24-BCS-068

N	BP = 600 + N	H = (N×7) mod 360	K = (N mod 3) + 3	Cart limit = (N mod 4) + 2	Delivery dates
68	668px	116 → hsl(116, 70%, 45%)	5	2	2026-10-29 to 2027-01-05

Project: Lama Retail replica (Assignment 2) copied to `midterm/` (commit "midterm: baseline copy"). Bootstrap 5.3.3 kept. Screenshots at 375px, BP = 668px and 1280px. Task 1 and 2 use no JavaScript of my own; Task 3 is in `script.js`.

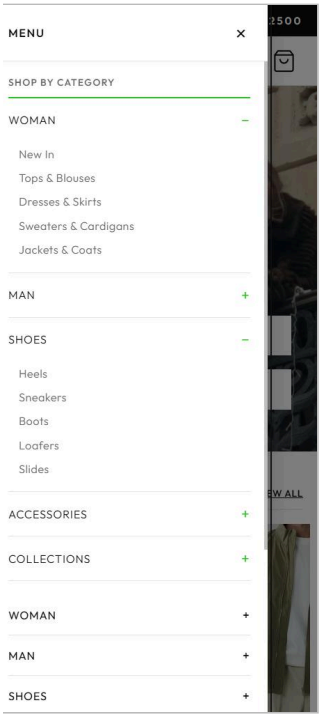
Live site (GitHub Pages): https://hammadsarwar789.github.io/WEB-DEV/
Only the `midterm/` folder is published (GitHub Actions workflow `.github/workflows/deploy-midterm.yml`).

Contents

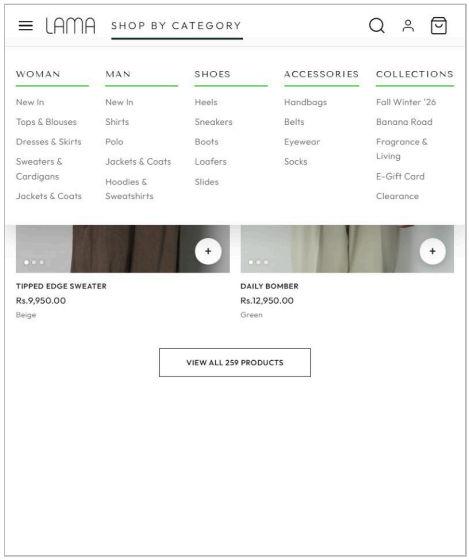
1. Screenshots – Task 1 (mega menu), Task 2 (checkout validation), Task 3 (cart badge + mini cart)
2. Files changed (every file added, changed or removed)
3. Code – Task 1, Task 2, Task 3, Extra (as text)
4. Live site – GitHub Pages deployment

Task 1 – Mega menu – screenshots

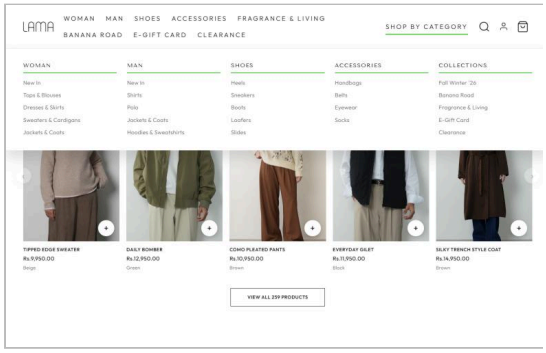
1280px and 668px: “Shop by Category” open (full-width panel, K = 5 columns, above the product grid). 375px: the same groups as a <details> accordion in the Bootstrap offcanvas menu (Woman and Shoes opened, the rest closed).



375px – accordion in hamburger menu



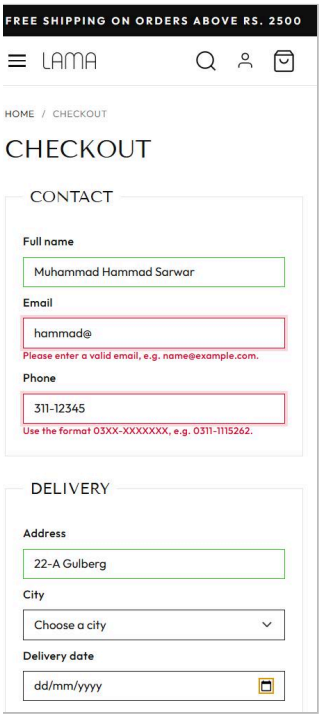
668px (BP) – panel open



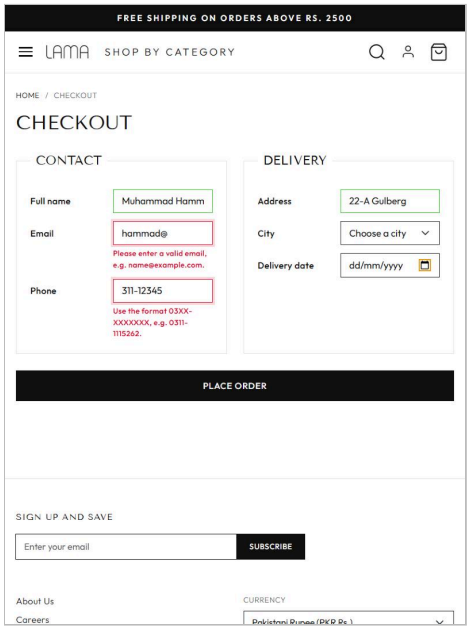
1280px – panel open

Task 2 – Checkout form – screenshots

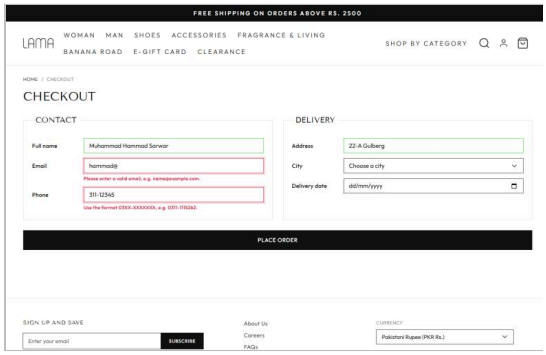
After typing an invalid email and phone: red border + message only on the touched invalid fields; valid fields get the green accent border; untouched fields stay neutral. 375px: one column, labels above inputs. 668px and 1280px: fieldsets side by side, labels in a left column. Submit button full width.



375px – one column



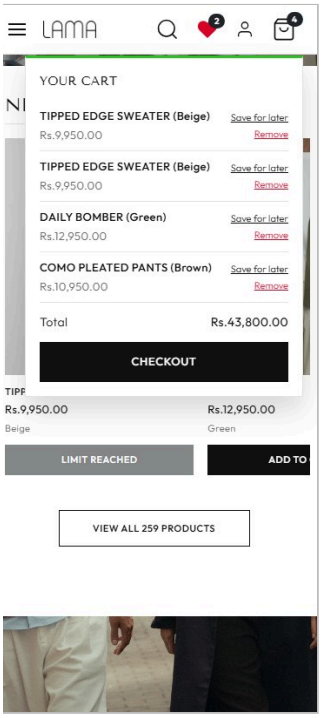
668px (BP) – side by side



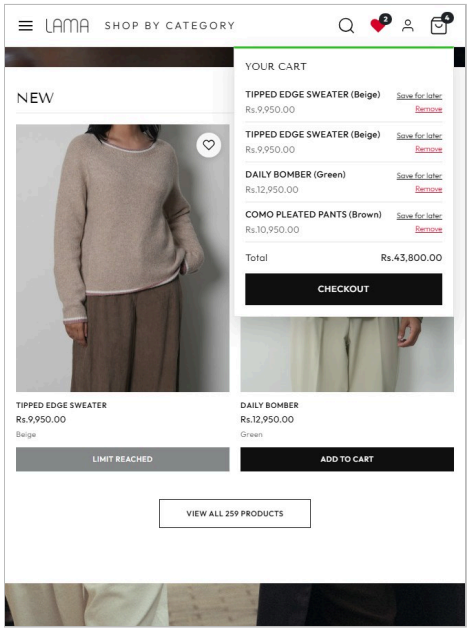
1280px – side by side

Task 3 – Cart badge and mini cart – screenshots

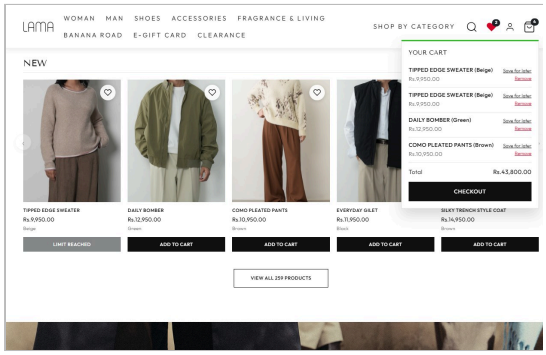
After using the cart: Tipped Edge Sweater added twice (button now “Limit reached”, disabled), Daily Bomber and Como Pleated Pants added once. Badge shows 4; mini cart lists each item with its price, Remove (and Save for later, the extra feature), total Rs.43,800.00 calculated with reduce().



375px – mini cart open



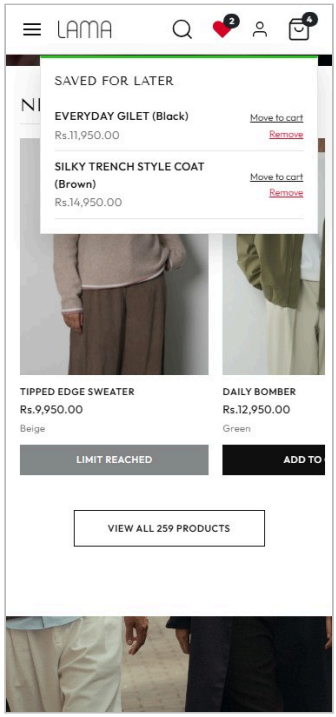
668px (BP) – mini cart open



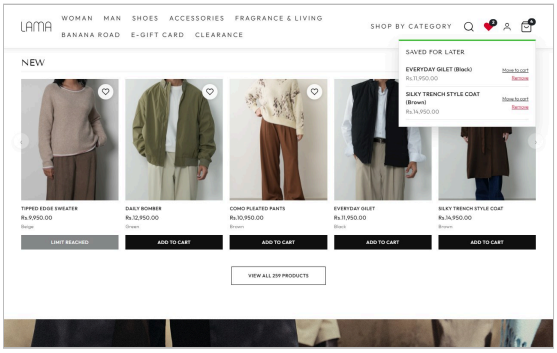
1280px – mini cart open

Extra (not required) – Save for later – screenshots

Hearts on the product cards save items for later (kept in localStorage). The header heart shows the count and opens the “Saved for later” panel with Move to cart and Remove. Each mini-cart row also has Save for later.



375px – saved panel



1280px – saved panel

Files changed

Compared with the baseline commit “midterm: baseline copy”. Every other file in `midterm/` is unchanged from Assignment 2.

File	Task	Change
`index.html`	1, 2, 3, Extra	Shop by Category menu + panel; phone-menu accordion; hamburger uses Bootstrap offcanvas; cart icon is a link to `checkout.html`; mini-cart panel; header heart + "Saved for later" panel; loads `css/mega-menu.css`, `css/mini-cart.css`, `css/wishlist.css`, Bootstrap JS bundle and `script.js`; old size picker and cart drawer removed
`product.html`	1, 2, 3	The same header changes as `index.html` (the header is shared)
`checkout.html`	2	**New** – the checkout form (same header / footer as the other pages)
`css/mega-menu.css`	1	**New** – mega-menu panel, `<details>` accordion, BP / K / H rules
`css/checkout.css`	2	**New** – form layout (CSS Grid) and `:user-invalid` validation styles
`css/mini-cart.css`	3	**New** – Add to Cart button and mini-cart panel styles
`script.js`	3, Extra	**New** – cart array, badge, mini cart, `reduce()` total, limit of 2; plus the save-for-later list
`js/menu.js`	1	Removed our own hamburger open/close code (Bootstrap's offcanvas script does it now)
`js/products.js`	3, Extra	Every product card gets an **Add to Cart** button instead of the "+" quick-add button, and a heart (save for later)
`css/wishlist.css`	Extra	**New** – hearts on cards, header heart and "Saved for later" panel styles
`README.md`	–	This file
`FA24-BCS-068-midterm.pdf`	–	**New** – submission PDF (live site link, screenshots, files changed, code for each task, deployment workflow)
`screenshots/midterm/` (11 files)	–	**New** – each task at 375px, 668px and 1280px + the saved-for-later panel, used in the PDF
`.github/workflows/deploy-midterm.yml` (repo root)	–	**New** – GitHub Actions workflow that publishes only this `midterm/` folder to GitHub Pages
`FA24-BCS-068_Assignment2_Completion.pdf`	–	**Removed** from `midterm/` (copied from Assignment 2) so this folder has only one PDF; the original is still in `assignments/assignment 2/`

Task 1 – code added / changed

index.html / product.html – "Shop by Category" item inside the existing <nav>

```
                    <!-- MIDTERM TASK 1: "Shop by Category" mega menu - HTML + CSS only (css/mega-menu.css).
                         Opens on :hover or :focus-within (Tab), from BP = 668px and wider. -->
                    <ul class="navbar-nav shop-nav">
                        <li class="nav-item shop-menu">
                            <a class="nav-link shop-menu__link" href="#shop-panel" aria-haspopup="true">Shop by Category</a>
                            <div class="shop-panel" id="shop-panel">
                                <div class="container shop-panel__grid">
                                    <div class="shop-panel__col">
                                        <h2 class="shop-panel__heading" id="shop-heading-woman">Woman</h2>
                                        <ul class="shop-panel__list" aria-labelledby="shop-heading-woman">
                                            <li><a href="index.html#section-new">New In</a></li>
                                            <li><a href="#">Tops &amp;amp; Blouses</a></li>
                                            <li><a href="#">Dresses &amp;amp; Skirts</a></li>
                                            <li><a href="index.html#section-sweaters">Sweaters &amp;amp; Cardigans</a></li>
                                            <li><a href="index.html#section-jackets">Jackets &amp;amp; Coats</a></li>
                                        </ul>
                                    </div>
                                    <div class="shop-panel__col">
                                        <h2 class="shop-panel__heading" id="shop-heading-man">Man</h2>
                                        <ul class="shop-panel__list" aria-labelledby="shop-heading-man">
                                            <li><a href="index.html#section-new">New In</a></li>
                                            <li><a href="#">Shirts</a></li>
                                            <li><a href="#">Polo</a></li>
                                            <li><a href="index.html#section-jackets">Jackets &amp;amp; Coats</a></li>
                                            <li><a href="#">Hoodies &amp;amp; Sweatshirts</a></li>
                                        </ul>
                                    </div>
                                    <div class="shop-panel__col">
                                        <h2 class="shop-panel__heading" id="shop-heading-shoes">Shoes</h2>
                                        <ul class="shop-panel__list" aria-labelledby="shop-heading-shoes">
                                            <li><a href="index.html#section-shoes">Heels</a></li>
                                            <li><a href="index.html#section-shoes">Sneakers</a></li>
                                            <li><a href="index.html#section-shoes">Boots</a></li>
                                            <li><a href="index.html#section-shoes">Loafers</a></li>
                                            <li><a href="#">Slides</a></li>
                                        </ul>
                                    </div>
                                    <div class="shop-panel__col">
                                        <h2 class="shop-panel__heading" id="shop-heading-accessories">Accessories</h2>
                                        <ul class="shop-panel__list" aria-labelledby="shop-heading-accessories">
                                            <li><a href="index.html#section-bags">Handbags</a></li>
                                            <li><a href="#">Belts</a></li>
                                            <li><a href="#">Eyewear</a></li>
                                            <li><a href="#">Socks</a></li>
                                        </ul>
                                    </div>
                                    <div class="shop-panel__col">
                                        <h2 class="shop-panel__heading" id="shop-heading-collections">Collections</h2>
                                        <ul class="shop-panel__list" aria-labelledby="shop-heading-collections">
                                            <li><a href="#">Fall Winter '26</a></li>
                                            <li><a href="#">Banana Road</a></li>
                                            <li><a href="#">Fragrance &amp;amp; Living</a></li>
                                            <li><a href="#">E-Gift Card</a></li>
                                            <li><a href="#">Clearance</a></li>
                                        </ul>
                                    </div>
                                </div>
                            </div>
                        </li>
                    </ul>
```

index.html / product.html – hamburger now opened by Bootstrap's own offcanvas script

```
                    <button type="button" id="mobileMenuBtn" class="mobile-menu-btn" aria-label="Open mobile navigation menu"
data-bs-toggle="offcanvas" data-bs-target="#mobileDrawer" aria-controls="mobileDrawer">
...
    <aside id="mobileDrawer" class="offcanvas offcanvas-start mobile-drawer" tabindex="-1" aria-label="Navigation Menu">
...
            <button type="button" id="mobileDrawerClose" class="mobile-drawer-close" data-bs-dismiss="offcanvas" aria-
label="Close menu">&times;</button>
...
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
```

```
            integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz" crossorigin="anonymous">
</script>
```

index.html / product.html – the same K = 5 groups as a <details> accordion (below BP)

```
            <!-- MIDTERM TASK 1: the same K = 5 groups as a <details> accordion (below BP = 668px only).
                 Closed by default; each <details> opens on its own. No JavaScript. -->
            <nav class="shop-accordion" aria-label="Shop by Category">
                <p class="shop-accordion__title">Shop by Category</p>
                <details class="shop-accordion__group">
                    <summary>Woman</summary>
                    <ul>
                        <li><a href="index.html#section-new">New In</a></li>
                        <li><a href="#">Tops &amp; Blouses</a></li>
                        <li><a href="#">Dresses &amp; Skirts</a></li>
                        <li><a href="index.html#section-sweaters">Sweaters &amp; Cardigans</a></li>
                        <li><a href="index.html#section-jackets">Jackets &amp; Coats</a></li>
                    </ul>
                </details>
                <details class="shop-accordion__group">
                    <summary>Man</summary>
                    <ul>
                        <li><a href="index.html#section-new">New In</a></li>
                        <li><a href="#">Shirts</a></li>
                        <li><a href="#">Polo</a></li>
                        <li><a href="index.html#section-jackets">Jackets &amp; Coats</a></li>
                        <li><a href="#">Hoodies &amp; Sweatshirts</a></li>
                    </ul>
                </details>
                <details class="shop-accordion__group">
                    <summary>Shoes</summary>
                    <ul>
                        <li><a href="index.html#section-shoes">Heels</a></li>
                        <li><a href="index.html#section-shoes">Sneakers</a></li>
                        <li><a href="index.html#section-shoes">Boots</a></li>
                        <li><a href="index.html#section-shoes">Loafers</a></li>
                        <li><a href="#">Slides</a></li>
                    </ul>
                </details>
                <details class="shop-accordion__group">
                    <summary>Accessories</summary>
                    <ul>
                        <li><a href="index.html#section-bags">Handbags</a></li>
                        <li><a href="#">Belts</a></li>
                        <li><a href="#">Eyewear</a></li>
                        <li><a href="#">Socks</a></li>
                    </ul>
                </details>
                <details class="shop-accordion__group">
                    <summary>Collections</summary>
                    <ul>
                        <li><a href="#">Fall Winter '26</a></li>
                        <li><a href="#">Banana Road</a></li>
                        <li><a href="#">Fragrance &amp; Living</a></li>
                        <li><a href="#">E-Gift Card</a></li>
                        <li><a href="#">Clearance</a></li>
                    </ul>
                </details>
            </nav>
```

css/mega-menu.css (new file, complete)

```
/* ============================================================================
   css/mega-menu.css - MIDTERM TASK 1: "Shop by Category" mega menu (HTML + CSS only)

   My numbers (roll no. FA24-BCS-068):  N = 68
     BP = 600 + N = 668px      -> the panel works from 668px; below it the <details> accordion is used
     K  = (N mod 3) + 3 = 5    -> 5 columns in the panel / 5 accordion groups
     H  = (N x 7) mod 360 = 116 -> accent colour hsl(116, 70%, 45%)

   To change BP: search this file for "BP" (CSS variables can't be used inside @media).
   To change K:  edit grid-template-columns in .shop-panel__grid.
   ============================================================================ */

:root {
    --accent: hsl(116, 70%, 45%);   /* H = 116 */
}
```

```
/* 1. The desktop <nav> now shows from BP (it used to show only from 1024px).
      Between BP and 1023px only "Shop by Category" is visible; the other 8 links stay in the hamburger. */
@media (min-width: 668px) {                    /* BP */
    .desktop-nav {
        display: flex;
        align-items: center;
        flex: 1;
        gap: var(--space-6);
        margin: 0 var(--space-4) 0 var(--space-6);
    }
}

@media (max-width: 1023.98px) {
    .desktop-nav .nav-list {
        display: none;                         /* the JS-built links still appear only from 1024px */
    }
}

/* 2. Position: the panel is placed relative to the sticky header (full page width),
      so the navbar and the menu item must NOT be the positioned parent. */
.header-inner.navbar {
    position: static;                          /* Bootstrap's .navbar is position: relative */
}

.shop-menu.nav-item {
    position: static;                          /* our .nav-item is position: relative */
}

/* 3. The panel: hidden by default, full width under the navbar, above everything */
.shop-panel {
    position: absolute;                        /* out of the page flow -> does not push content down */
    top: 100%;                                 /* starts at the bottom edge of the header */
    left: 0;
    right: 0;                                  /* left 0 + right 0 = full width of the header */
    z-index: 10;                               /* inside the header, which is z-index 500 above the page */
    background-color: var(--color-white);
    border-top: 1px solid var(--color-border);
    box-shadow: 0 18px 30px rgba(0, 0, 0, 0.12);
    padding: var(--space-8) 0;

    visibility: hidden;                        /* hidden = not clickable and not reachable with Tab */
    opacity: 0;
    /* When CLOSING, wait 0.25s before hiding: the pointer has time to travel from
       "Shop by Category" across the gap into the panel without it closing. */
    transition: opacity 0.15s ease, visibility 0s linear 0.25s;
}

/* OPEN: mouse over the item or the panel (:hover), or keyboard focus anywhere inside it (:focus-within).
   The panel is inside the <li>, so hovering the panel still counts as hovering the <li>. */
.shop-menu:hover > .shop-panel,
.shop-menu:focus-within > .shop-panel {
    visibility: visible;
    opacity: 1;
    transition-delay: 0s;                      /* open immediately */
}

/* Below BP the panel never shows (the accordion in the hamburger menu is used instead) */
@media (max-width: 667.98px) {                 /* BP - 0.02 */
    .shop-nav {
        display: none;
    }
}

/* 4. K columns, each a list with a heading */
.shop-panel__grid {
    display: grid;
    grid-template-columns: repeat(5, 1fr);     /* K = 5 */
    gap: var(--space-6);
}

.shop-panel__heading {
    font-size: var(--font-size-sm);
    text-transform: uppercase;
    letter-spacing: var(--letter-spacing-wider);
    padding-bottom: var(--space-2);
    margin-bottom: var(--space-3);
    border-bottom: 2px solid var(--accent);    /* accent colour from H */
}

.shop-panel__list li + li {
```

```
    margin-top: var(--space-2);
}

.shop-panel__list a {
    font-size: var(--font-size-sm);
    color: var(--color-text-muted);
}

.shop-panel__list a:hover,
.shop-panel__list a:focus-visible {
    color: var(--color-text-main);
    text-decoration: underline;
    text-decoration-color: var(--accent);
}

.shop-menu:hover > .shop-menu__link,
.shop-menu:focus-within > .shop-menu__link {
    box-shadow: inset 0 -2px 0 var(--accent);   /* accent underline while the panel is open */
}

/* 5. Below BP: the same K groups as a <details> accordion inside the hamburger menu */
.shop-accordion {
    margin-bottom: var(--space-4);
}

@media (min-width: 668px) {                     /* BP */
    .shop-accordion {
        display: none;                          /* from BP the desktop panel is used instead */
    }
}

.shop-accordion__title {
    font-size: var(--font-size-xs);
    font-weight: 500;
    text-transform: uppercase;
    letter-spacing: var(--letter-spacing-wider);
    color: var(--color-text-muted);
    padding-bottom: var(--space-2);
    border-bottom: 2px solid var(--accent);
}

.shop-accordion__group {
    border-bottom: 1px solid var(--color-border);
}

.shop-accordion__group summary {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: var(--space-3) 0;
    font-size: var(--font-size-sm);
    text-transform: uppercase;
    letter-spacing: var(--letter-spacing-wide);
    cursor: pointer;
    list-style: none;                           /* hide the default triangle (Firefox) */
}

.shop-accordion__group summary::-webkit-details-marker {
    display: none;                              /* hide the default triangle (Chrome / Safari) */
}

.shop-accordion__group summary::after {
    content: '+';
    color: var(--accent);
    font-size: 1.1rem;
}

.shop-accordion__group[open] summary::after {
    content: '\2212';                           /* minus sign when open */
}

.shop-accordion__group ul {
    padding: 0 0 var(--space-3) var(--space-3);
}

.shop-accordion__group li a {
    display: block;
    padding: var(--space-1) 0;
    font-size: var(--font-size-sm);
    color: var(--color-text-muted);
```

```
}

/* 6. Phone menu = Bootstrap offcanvas now: let Bootstrap's z-index and slide animation apply
      (our old .mobile-drawer rule set a lower z-index meant for the removed custom backdrop) */
.mobile-drawer.offcanvas {
    z-index: var(--bs-offcanvas-zindex);
    transition: transform 0.3s ease-in-out;
}
```

js/menu.js – our own hamburger code removed (Bootstrap does it now); replaced by this comment

```
        // 3. Mobile Drawer: opened and closed by Bootstrap's own offcanvas script (data-bs-toggle="offcanvas"
        //    on the hamburger button in the HTML), which also handles Esc, the backdrop and focus. (Midterm Task 1)
```

Task 2 – code added / changed

index.html / product.html – cart icon is now a link to checkout.html

```
                    <!-- MIDTERM TASK 2: the cart icon is a real link to the checkout page -->
                    <a href="checkout.html" id="cartTriggerBtn" class="header-icon-btn" aria-label="View cart and checkout">
```

checkout.html – <main> with the form (header and footer are the same as the other pages)

```
         MIDTERM TASK 2: CHECKOUT FORM - HTML5 validation + CSS only (css/checkout.css), no JavaScript
         ========================================================================== -->
    <main id="MainContent" role="main" class="checkout-page">
        <div class="container">

            <nav aria-label="Breadcrumb">
                <ol class="breadcrumb checkout-breadcrumb">
                    <li class="breadcrumb-item"><a href="index.html">Home</a></li>
                    <li class="breadcrumb-item active" aria-current="page">Checkout</li>
                </ol>
            </nav>

            <h1 class="checkout-title">Checkout</h1>

            <!-- Shown only after a valid submit: the form goes to #order-confirmed and CSS :target shows this box -->
            <p id="order-confirmed" class="order-confirmed" role="status">
                Thank you! Your order details were accepted. (Demo form &amp;ndash; nothing is sent anywhere.)
            </p>

            <!-- The browser checks required / type / pattern / minlength / min / max before it lets the form submit.
                 Inputs have no name attribute, so no personal data is added to the URL. -->
            <form class="checkout-form" action="#order-confirmed">

                <fieldset class="checkout-fieldset">
                    <legend>Contact</legend>
                        <label for="full-name">Full name</label>
                        <input id="full-name" class="form-control" type="text" required minlength="3" autocomplete="name"
placeholder="e.g. Muhammad Hammad Sarwar" aria-describedby="full-name-error">
                        <span class="field-error" id="full-name-error">Please enter your full name (at least 3 letters).</span>
                        <label for="email">Email</label>
                        <input id="email" class="form-control" type="email" required autocomplete="email"
placeholder="name@example.com" aria-describedby="email-error">
                        <span class="field-error" id="email-error">Please enter a valid email, e.g. name@example.com.</span>
                        <label for="phone">Phone</label>
                        <input id="phone" class="form-control" type="tel" required pattern="03[0-9]{2}-[0-9]{7}"
placeholder="03XX-XXXXXXX" title="Format: 03XX-XXXXXXX" autocomplete="tel" aria-describedby="phone-error">
                        <span class="field-error" id="phone-error">Use the format 03XX-XXXXXXX, e.g. 0311-1115262.</span>
                </fieldset>

                <fieldset class="checkout-fieldset">
                    <legend>Delivery</legend>
                        <label for="address">Address</label>
                        <input id="address" class="form-control" type="text" required minlength="5" autocomplete="street-
address" placeholder="House, street, area" aria-describedby="address-error">
                        <span class="field-error" id="address-error">Please enter your street address.</span>
                        <label for="city">City</label>
                        <select id="city" class="form-select" required aria-describedby="city-error">
                            <option value="">Choose a city</option>
                            <option>Lahore</option>
                            <option>Karachi</option>
                            <option>Islamabad</option>
                            <option>Rawalpindi</option>
                            <option>Faisalabad</option>
                            <option>Multan</option>
                            <option>Peshawar</option>
                        </select>
                        <span class="field-error" id="city-error">Please choose a city.</span>
                        <label for="delivery-date">Delivery date</label>
                        <input id="delivery-date" class="form-control" type="date" required min="2026-10-29" max="2027-01-05"
aria-describedby="delivery-date-error">
                        <span class="field-error" id="delivery-date-error">Choose a date from 29 Oct 2026 to 5 Jan 2027.</span>
                </fieldset>

                <button type="submit" class="btn btn-dark checkout-submit">Place order</button>
            </form>
```

```
        </div>
    </main>
```

css/checkout.css (new file, complete)

```
/* ========================================================================
   css/checkout.css - MIDTERM TASK 2: checkout form (HTML5 validation + CSS only, no JavaScript)

   BP = 668px (search "BP" to change it)
     below BP : one column, labels above inputs
     from BP  : the two fieldsets side by side; inside each, labels in a left column (CSS Grid)
   ======================================================================== */

.checkout-page {
    padding: var(--space-6) 0 var(--space-12);
}

.checkout-breadcrumb {
    font-size: var(--font-size-xs);
    text-transform: uppercase;
    letter-spacing: var(--letter-spacing-wide);
    margin-bottom: var(--space-4);
}

.checkout-title {
    font-size: var(--font-size-2xl);
    text-transform: uppercase;
    margin-bottom: var(--space-6);
}

/* 1. Page layout: the form is a grid - 1 column on phones, 2 columns (Contact | Delivery) from BP */
.checkout-form {
    display: grid;
    grid-template-columns: 1fr;
    gap: var(--space-6);
}

@media (min-width: 668px) {                     /* BP */
    .checkout-form {
        grid-template-columns: 1fr 1fr;         /* the two fieldsets sit side by side */
    }
}

/* Submit button: full width at every size, under both fieldsets */
.checkout-submit {
    grid-column: 1 / -1;                        /* span all columns of the form grid */
    width: 100%;
}

/* 2. Each fieldset is its own grid: label | input (+ error message under the input) */
.checkout-fieldset {
    display: grid;
    grid-template-columns: 1fr;                 /* below BP: label above input */
    row-gap: var(--space-2);
    align-items: center;
    align-content: start;
    border: 1px solid var(--color-border);
    padding: var(--space-5);
    margin: 0;
}

@media (min-width: 668px) {                     /* BP */
    .checkout-fieldset {
        grid-template-columns: 6.5rem minmax(0, 1fr);   /* labels in a column on the left of their inputs */
        column-gap: var(--space-4);
        row-gap: var(--space-3);
    }

    .checkout-fieldset .field-error {
        grid-column: 2;                         /* message sits under its input, not under the label */
    }
}

.checkout-fieldset legend {
    float: none;
    width: auto;
    padding: 0 var(--space-2);
    margin-bottom: var(--space-2);
    font-family: var(--font-heading);
```

```
    font-size: var(--font-size-lg);
    text-transform: uppercase;
}

.checkout-fieldset label {
    font-size: var(--font-size-sm);
    font-weight: 500;
}

.checkout-fieldset .form-control,
.checkout-fieldset .form-select {
    font-size: var(--font-size-base);
    border-color: var(--color-border-dark);
}

.checkout-fieldset .form-control:focus,
.checkout-fieldset .form-select:focus {
    border-color: var(--accent);               /* accent colour hsl(116, 70%, 45%) from css/mega-menu.css */
    box-shadow: 0 0 0 0.2rem hsl(116 70% 45% / 0.25);
}

/* 3. Validation messages: hidden until the user has interacted with the field */
.field-error {
    display: none;
    color: var(--color-sale);
    font-size: var(--font-size-xs);
    font-weight: 500;
    margin-top: calc(var(--space-2) * -1);
}

/* :user-invalid = invalid AND the user has already typed in / left the field (not on page load) */
.checkout-form .form-control:user-invalid,
.checkout-form .form-select:user-invalid {
    border-color: var(--color-sale);           /* red border */
    box-shadow: 0 0 0 0.2rem rgba(217, 4, 41, 0.15);
}

.checkout-form .form-control:user-invalid + .field-error,
.checkout-form .form-select:user-invalid + .field-error {
    display: block;                            /* "+" = the message right after the invalid field */
}

/* Valid fields the user filled in get a green (accent) border */
.checkout-form .form-control:user-valid,
.checkout-form .form-select:user-valid {
    border-color: var(--accent);
}

/* Fallback for older browsers without :user-invalid:
   :placeholder-shown is true while the field is empty, so this only fires after typing */
@supports not selector(:user-invalid) {
    .checkout-form .form-control:not(:placeholder-shown):invalid {
        border-color: var(--color-sale);
    }

    .checkout-form .form-control:not(:placeholder-shown):invalid + .field-error {
        display: block;
    }
}

/* 4. Confirmation box: shown only when the URL ends with #order-confirmed (after a valid submit) */
.order-confirmed {
    display: none;
    padding: var(--space-4);
    margin-bottom: var(--space-6);
    border: 1px solid var(--accent);
    border-left-width: 4px;
    background-color: hsl(116 70% 45% / 0.08);
    font-weight: 500;
}

.order-confirmed:target {
    display: block;
}
```

Task 3 – code added / changed

script.js (new file, complete)

```
/**
 * script.js - MIDTERM TASK 3: cart badge + mini cart (plain JavaScript, no libraries)
 *             + EXTRA: "Save for later" (wishlist) with hearts on the cards
 *
 * N = 68  ->  each product may be added at most (N mod 4) + 2 = (68 mod 4) + 2 = 0 + 2 = 2 times
 */
(function () {
    "use strict";

    const MAX_PER_PRODUCT = (68 % 4) + 2;   // = 2
    const SAVED_STORAGE_KEY = "lama_saved"; // the saved list is kept in localStorage (survives a refresh)

    // The cart: an array of { key, name, colour, price } objects (one entry per item added)
    let cart = [];

    // Saved for later: an array of { key, name, colour, price } (each product at most once)
    let saved = loadSaved();

    // "Rs.9,950.00" -> 9950   (remove the "Rs." prefix and the thousands commas)
    function parsePrice(text) {
        return Number(text.replace("Rs.", "").replace(/,/g, "").trim());
    }

    // 9950 -> "Rs.9,950.00"
    function formatPrice(n) {
        return "Rs." + n.toLocaleString("en-PK", { minimumFractionDigits: 2, maximumFractionDigits: 2 });
    }

    // Same product = same name AND colour (e.g. Short Knit Polo comes in Brown and Light Blue)
    function makeKey(name, colour) {
        return name + "|" + colour;
    }

    // How many of this product are in the cart?
    function countOf(key) {
        return cart.filter(item => item.key === key).length;
    }

    function isSaved(key) {
        return saved.some(item => item.key === key);
    }

    // localStorage can contain broken data or be blocked, so read it safely
    function loadSaved() {
        try {
            const list = JSON.parse(localStorage.getItem(SAVED_STORAGE_KEY));
            return Array.isArray(list) ? list.filter(item => item && item.key && typeof item.price === "number") : [];
        } catch (err) {
            return [];
        }
    }

    function storeSaved() {
        try {
            localStorage.setItem(SAVED_STORAGE_KEY, JSON.stringify(saved));
        } catch (err) {
            // storage blocked - the list still works until the page is closed
        }
    }

    document.addEventListener("DOMContentLoaded", function () {
        const cartIcon = document.getElementById("cartTriggerBtn");
        const badge = document.getElementById("headerCartBadge");
        const panel = document.getElementById("miniCart");
        const list = document.getElementById("miniCartList");
        const emptyText = document.getElementById("miniCartEmpty");
        const totalEl = document.getElementById("miniCartTotal");

        const savedBtn = document.getElementById("savedBtn");
        const savedBadge = document.getElementById("savedBadge");
        const savedPanel = document.getElementById("savedPanel");
        const savedList = document.getElementById("savedList");
        const savedEmpty = document.getElementById("savedEmpty");
        if (!cartIcon || !badge || !panel) return;
```

```
cartIcon.setAttribute("aria-controls", "miniCart");
cartIcon.setAttribute("aria-expanded", "false");

// ---------- Add an item (respecting the limit), then redraw everything
function addItem(name, colour, price) {
    const key = makeKey(name, colour);
    if (countOf(key) >= MAX_PER_PRODUCT) return false;
    cart.push({ key: key, name: name, colour: colour, price: price });
    render();
    return true;
}

// ---------- Remove one item by its position in the array
function removeItem(index) {
    cart.splice(index, 1);
    render();
}

// ---------- Save for later: add / remove a product in the saved list
function toggleSaved(name, colour, price) {
    const key = makeKey(name, colour);
    if (isSaved(key)) {
        saved = saved.filter(item => item.key !== key);
    } else {
        saved.push({ key: key, name: name, colour: colour, price: price });
    }
    storeSaved();
    render();
}

// From the mini cart: take the item out of the cart and keep it in the saved list
function saveForLater(index) {
    const item = cart[index];
    cart.splice(index, 1);
    if (!isSaved(item.key)) saved.push({ key: item.key, name: item.name, colour: item.colour, price: item.price });
    storeSaved();
    render();
}

// From the saved panel: put it in the cart (if under the limit) and take it off the saved list
function moveToCart(index) {
    const item = saved[index];
    if (addItem(item.name, item.colour, item.price)) {
        saved.splice(index, 1);
        storeSaved();
        render();
        return true;
    }
    return false;   // limit reached - it stays saved
}

// Small helper: a text button used in both panels
function textButton(label, className, ariaLabel, onClick) {
    const btn = document.createElement("button");
    btn.type = "button";
    btn.className = className;
    btn.textContent = label;
    btn.setAttribute("aria-label", ariaLabel);
    btn.addEventListener("click", function (e) {
        e.stopPropagation();      // keep the panel open
        onClick(btn);
    });
    return btn;
}

// ---------- Redraw: badges, both panels, totals, and every card button
function render() {
    // Cart badge = number of items in the cart (CSS hides it when data-count is "0")
    badge.textContent = cart.length;
    badge.setAttribute("data-count", cart.length);
    cartIcon.setAttribute("aria-label", `View cart, ${cart.length} item${cart.length === 1 ? "" : "s"}`);

    // Mini-cart list
    list.innerHTML = "";
    cart.forEach(function (item, index) {
        const li = document.createElement("li");
        li.className = "mini-cart__item";

        const info = document.createElement("span");
```

```
                info.className = "mini-cart__name";
                info.textContent = `${item.name} (${item.colour})`;

                const price = document.createElement("span");
                price.className = "mini-cart__price";
                price.textContent = formatPrice(item.price);

                const actions = document.createElement("span");
                actions.className = "mini-cart__actions";
                actions.append(
                    textButton("Save for later", "mini-cart__save", `Save ${item.name} (${item.colour}) for later`, () =>
saveForLater(index)),
                    textButton("Remove", "mini-cart__remove", `Remove ${item.name} (${item.colour})`, () => removeItem(index))
                );

                li.append(info, price, actions);
                list.appendChild(li);
            });
            emptyText.hidden = cart.length > 0;

            // Total with reduce(): start at 0 and add each item's price
            const total = cart.reduce((sum, item) => sum + item.price, 0);
            totalEl.textContent = formatPrice(total);

            // Add to Cart buttons: "Limit reached" + disabled once a product is in the cart MAX_PER_PRODUCT times
            document.querySelectorAll(".btn-add-to-cart").forEach(function (btn) {
                const card = btn.closest(".product-card");
                const key = makeKey(cardName(card), cardColour(card));
                const full = countOf(key) >= MAX_PER_PRODUCT;
                btn.disabled = full;
                btn.textContent = full ? "Limit reached" : "Add to Cart";
            });

            renderSaved();
        }

        // ---------- Saved-for-later panel, heart buttons and the saved badge
        function renderSaved() {
            if (savedBadge) {
                savedBadge.textContent = saved.length;
                savedBadge.setAttribute("data-count", saved.length);
            }
            if (savedBtn) savedBtn.setAttribute("aria-label", `Saved for later, ${saved.length} item${saved.length === 1 ? "" :
"s"}`);

            // Hearts on the cards: filled (aria-pressed="true") when the product is saved
            document.querySelectorAll(".btn-wishlist").forEach(function (heart) {
                const card = heart.closest(".product-card");
                const name = cardName(card);
                const on = isSaved(makeKey(name, cardColour(card)));
                heart.setAttribute("aria-pressed", on ? "true" : "false");
                heart.setAttribute("aria-label", on ? `Remove ${name} from saved items` : `Save ${name} for later`);
            });

            if (!savedList) return;
            savedList.innerHTML = "";
            saved.forEach(function (item, index) {
                const li = document.createElement("li");
                li.className = "mini-cart__item";

                const info = document.createElement("span");
                info.className = "mini-cart__name";
                info.textContent = `${item.name} (${item.colour})`;

                const price = document.createElement("span");
                price.className = "mini-cart__price";
                price.textContent = formatPrice(item.price);

                const actions = document.createElement("span");
                actions.className = "mini-cart__actions";
                actions.append(
                    textButton("Move to cart", "mini-cart__save", `Move ${item.name} (${item.colour}) to the cart`, function
(btn) {
                        if (!moveToCart(index)) {
                            btn.textContent = "Limit reached";
                            btn.disabled = true;
                        }
                    }),
                    textButton("Remove", "mini-cart__remove", `Remove ${item.name} (${item.colour}) from saved items`, function
() {
```

```
                saved.splice(index, 1);
                storeSaved();
                render();
            })
        );

        li.append(info, price, actions);
        savedList.appendChild(li);
    });
    savedEmpty.hidden = saved.length > 0;
}

// Read the product details from the card itself
function cardName(card) {
    return card.querySelector(".product-title").textContent.trim();
}

function cardColour(card) {
    const el = card.querySelector(".product-color-label");
    return el ? el.textContent.trim() : "";
}

function cardPrice(card) {
    return parsePrice(card.querySelector(".product-price").textContent);
}

// ---------- Clicks on any card button (event delegation: one listener for all cards,
//            including cards added later)
document.addEventListener("click", function (e) {
    const addBtn = e.target.closest(".btn-add-to-cart");
    if (addBtn && !addBtn.disabled) {
        const card = addBtn.closest(".product-card");
        addItem(cardName(card), cardColour(card), cardPrice(card));
        return;
    }

    const heart = e.target.closest(".btn-wishlist");
    if (heart) {
        const card = heart.closest(".product-card");
        toggleSaved(cardName(card), cardColour(card), cardPrice(card));
    }
});

// ---------- Product page: its own Add to Cart form sends a "cart:add" event (js/product-page.js)
window.addEventListener("cart:add", function (e) {
    const d = e.detail || {};
    const qty = Number.isInteger(d.quantity) ? d.quantity : 1;
    for (let i = 0; i < qty; i++) {
        if (!addItem(d.name, d.color, d.price)) break;   // stop at the limit
    }
    openPanel(panel, cartIcon);
});

// ---------- Panels: the cart icon toggles the mini cart, the heart toggles the saved panel.
//            Only one is open at a time.
function openPanel(which, button) {
    closeAll();
    which.hidden = false;
    button.setAttribute("aria-expanded", "true");
}

function closeAll() {
    panel.hidden = true;
    cartIcon.setAttribute("aria-expanded", "false");
    if (savedPanel) {
        savedPanel.hidden = true;
        savedBtn.setAttribute("aria-expanded", "false");
    }
}

cartIcon.addEventListener("click", function (e) {
    e.preventDefault();               // the icon is a link to checkout.html (Task 2); JS shows the mini cart instead
    if (panel.hidden) {
        openPanel(panel, cartIcon);
    } else {
        closeAll();
    }
});

if (savedBtn && savedPanel) {
```

```
            savedBtn.addEventListener("click", function () {
                if (savedPanel.hidden) {
                    openPanel(savedPanel, savedBtn);
                } else {
                    closeAll();
                }
            });
        }

        // Close with Esc or a click outside the panels
        document.addEventListener("keydown", function (e) {
            if (e.key === "Escape") closeAll();
        });

        document.addEventListener("click", function (e) {
            const insidePanel = panel.contains(e.target) || (savedPanel && savedPanel.contains(e.target));
            const onButton = cartIcon.contains(e.target) || (savedBtn && savedBtn.contains(e.target));
            if (!insidePanel && !onButton) closeAll();
        });

        render();
    });
})();
```

index.html / product.html – mini-cart panel under the cart icon

```
                    <!-- MIDTERM TASK 3: mini cart panel under the cart icon (filled and opened by script.js) -->
                    <div id="miniCart" class="mini-cart" role="region" aria-label="Mini cart" hidden>
                        <p class="mini-cart__title">Your cart</p>
                        <p id="miniCartEmpty" class="mini-cart__empty">Your cart is empty.</p>
                        <ul id="miniCartList" class="mini-cart__list"></ul>
                        <p class="mini-cart__total"><span>Total</span> <strong id="miniCartTotal">Rs.0.00</strong></p>
                        <a href="checkout.html" class="btn btn-dark w-100 mini-cart__checkout">Checkout</a>
                    </div>
```

index.html / product.html – script.js linked (old cart.js / quick-add.js removed)

```
    <script src="script.js"></script>                  <!-- Midterm Task 3: cart badge + mini cart -->
```

js/products.js – Add to Cart button added to every product card (the "+" quick-add button removed)

```
        // "Add to Cart" button (midterm Task 3): script.js listens for clicks on .btn-add-to-cart
        // and reads the product name and price from this card
        const addBtn = document.createElement("button");
        addBtn.type = "button";
        addBtn.className = "btn btn-dark btn-sm w-100 btn-add-to-cart";
        addBtn.textContent = "Add to Cart";
        meta.appendChild(addBtn);

        card.appendChild(meta);
```

css/mini-cart.css (new file, complete)

```
/* ========================================================================
   css/mini-cart.css - MIDTERM TASK 3: "Add to Cart" buttons on cards + mini-cart panel (script.js)
   ======================================================================== */

/* Add to Cart button at the bottom of every product card */
.btn-add-to-cart {
    margin-top: var(--space-2);
    padding: 8px 12px;
    font-size: var(--font-size-xs);
}

.btn-add-to-cart:disabled {
    opacity: 0.55;                      /* "Limit reached" */
}

/* The panel hangs under the cart icon (the icon group is the positioned parent) */
.header-right {
    position: relative;
}

.mini-cart {
    position: absolute;
    top: calc(100% + var(--space-2));
```

```
    right: 0;
    width: 320px;
    max-width: calc(100vw - 2rem);
    z-index: 20;                          /* inside the sticky header, above the page */
    background-color: var(--color-white);
    border: 1px solid var(--color-border);
    border-top: 3px solid var(--accent); /* accent colour hsl(116, 70%, 45%) */
    box-shadow: 0 16px 30px rgba(0, 0, 0, 0.15);
    padding: var(--space-4);
}

.mini-cart[hidden] {
    display: none;
}

.mini-cart__title {
    font-family: var(--font-heading);
    text-transform: uppercase;
    font-size: var(--font-size-base);
    margin-bottom: var(--space-3);
}

.mini-cart__empty {
    font-size: var(--font-size-sm);
    color: var(--color-text-muted);
    margin-bottom: var(--space-3);
}

.mini-cart__list {
    max-height: 260px;
    overflow-y: auto;
}

/* Each row: name | price | Remove  (CSS Grid) */
.mini-cart__item {
    display: grid;
    grid-template-columns: 1fr auto;
    gap: 2px var(--space-3);
    align-items: center;
    padding: var(--space-2) 0;
    border-bottom: 1px solid var(--color-border);
    font-size: var(--font-size-sm);
}

.mini-cart__name {
    font-weight: 500;
}

.mini-cart__price {
    color: var(--color-text-muted);
    grid-column: 1;
}

.mini-cart__remove {
    grid-column: 2;
    grid-row: 1 / 3;
    font-size: var(--font-size-xs);
    text-decoration: underline;
    color: var(--color-sale);
}

.mini-cart__total {
    display: flex;
    justify-content: space-between;
    margin: var(--space-3) 0;
    font-size: var(--font-size-base);
}
```

Extra (not required) – Save for later – code added

index.html / product.html – header heart (saved count) and the "Saved for later" panel

```
                    <!-- EXTRA: Save for later (wishlist) - heart with a count; opens the saved panel (script.js) -->
                    <button type="button" id="savedBtn" class="header-icon-btn saved-btn" aria-label="Saved for later, 0 items"
aria-controls="savedPanel" aria-expanded="false">
                        <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" aria-hidden="true"><path d="M20.8 4.6a5.5
5.5 0 0 0-7.8 0L12 5.7l-1-1.1a5.5 5.5 0 0 0-7.8 7.8l1 1.1L12 21l7.8-7.5 1-1.1a5.5 5.5 0 0 0 0-7.8z"></path></svg>
                        <span id="savedBadge" class="cart-count-badge badge rounded-pill text-bg-dark" data-count="0">0</span>
                    </button>
...
                    <!-- EXTRA: "Saved for later" panel (filled and opened by script.js; saved in localStorage) -->
                    <div id="savedPanel" class="mini-cart saved-panel" role="region" aria-label="Saved for later" hidden>
                        <p class="mini-cart__title">Saved for later</p>
                        <p id="savedEmpty" class="mini-cart__empty">Nothing saved yet. Tap the heart on a product to save it.
</p>
                        <ul id="savedList" class="mini-cart__list"></ul>
                    </div>
```

js/products.js – heart button on every product card

```
        // Heart = "Save for later" (extra feature): script.js handles clicks on .btn-wishlist
        const heartBtn = document.createElement("button");
        heartBtn.type = "button";
        heartBtn.className = "btn-wishlist";
        heartBtn.setAttribute("aria-pressed", "false");
        heartBtn.setAttribute("aria-label", `Save ${product.name} for later`);
        heartBtn.innerHTML = '<svg viewBox="0 0 24 24" aria-hidden="true"><path d="M20.8 4.6a5.5 5.5 0 0 0-7.8 0L12 5.7l-1-
1.1a5.5 5.5 0 0 0-7.8 7.8l1 1.1L12 21l7.8-7.5 1-1.1a5.5 5.5 0 0 0 0-7.8z"></path></svg>';
        imageBox.appendChild(heartBtn);
```

script.js – save-for-later functions (the complete file is in the Task 3 code above)

```
        // ---------- Save for later: add / remove a product in the saved list
        function toggleSaved(name, colour, price) {
            const key = makeKey(name, colour);
            if (isSaved(key)) {
                saved = saved.filter(item => item.key !== key);
            } else {
                saved.push({ key: key, name: name, colour: colour, price: price });
            }
            storeSaved();
            render();
        }

        // From the mini cart: take the item out of the cart and keep it in the saved list
        function saveForLater(index) {
            const item = cart[index];
            cart.splice(index, 1);
            if (!isSaved(item.key)) saved.push({ key: item.key, name: item.name, colour: item.colour, price: item.price });
            storeSaved();
            render();
        }

        // From the saved panel: put it in the cart (if under the limit) and take it off the saved list
        function moveToCart(index) {
            const item = saved[index];
            if (addItem(item.name, item.colour, item.price)) {
                saved.splice(index, 1);
                storeSaved();
                render();
                return true;
            }
            return false;   // limit reached - it stays saved
        }
```

css/wishlist.css (new file, complete)

```
/* ========================================================================
   css/wishlist.css - EXTRA: "Save for later" (hearts on cards, header heart, saved panel)
   The saved panel reuses the .mini-cart styles from css/mini-cart.css.
   ======================================================================== */

/* Heart on each product card (top-right corner of the photo) */
```

```css
.btn-wishlist {
    position: absolute;
    top: var(--space-3);
    right: var(--space-3);
    z-index: 5;
    width: 36px;
    height: 36px;
    border-radius: 50%;
    background-color: rgba(255, 255, 255, 0.92);
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.15);
    display: flex;
    align-items: center;
    justify-content: center;
    transition: transform var(--transition-fast);
}

.btn-wishlist:hover {
    transform: scale(1.08);
}

.btn-wishlist svg {
    width: 18px;
    height: 18px;
    fill: none;
    stroke: var(--color-primary);
    stroke-width: 1.8;
}

/* Saved = filled red heart (the state comes from aria-pressed, set by script.js) */
.btn-wishlist[aria-pressed="true"] svg {
    fill: var(--color-sale);
    stroke: var(--color-sale);
}

/* Header heart: same size as the other header icons, filled when something is saved */
.saved-btn svg {
    width: 24px;
    height: 24px;
}

.saved-btn:has(.cart-count-badge:not([data-count="0"])) svg {
    fill: var(--color-sale);
    stroke: var(--color-sale);
}

/* Row buttons in both panels: "Save for later" / "Move to cart" next to "Remove" */
.mini-cart__actions {
    grid-column: 2;
    grid-row: 1 / 3;
    display: flex;
    flex-direction: column;
    align-items: flex-end;
    gap: 2px;
}

.mini-cart__actions .mini-cart__remove {
    grid-column: auto;
    grid-row: auto;
}

.mini-cart__save {
    font-size: var(--font-size-xs);
    text-decoration: underline;
    color: var(--color-text-main);
    white-space: nowrap;
}

.mini-cart__save:disabled {
    color: var(--color-text-light);
    text-decoration: none;
}
```

Live site – GitHub Pages

URL: https://hammadsarwar789.github.io/WEB-DEV/

- Free hosting on GitHub Pages. Only the `midterm/` folder is published; the rest of the repository is not on the site.
- Repo Settings → Pages → Source: **GitHub Actions**. The workflow below uploads `midterm/` and deploys it.
- Every push that changes a file in `midterm/` redeploys the site automatically (about one minute).
- All links and asset paths are relative, so the site works from the `/WEB-DEV/` sub-path without changes.

.github/workflows/deploy-midterm.yml (new file, complete)

```
# Publishes ONLY the midterm/ folder to GitHub Pages.
# Site: https://hammadsarwar789.github.io/WEB-DEV/
# One-time setup: repo Settings -> Pages -> Source: "GitHub Actions".
name: Deploy midterm site

on:
  push:
    branches: [main]
    paths:
      - "midterm/**"
      - ".github/workflows/deploy-midterm.yml"
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: false

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/configure-pages@v5
      - uses: actions/upload-pages-artifact@v3
        with:
          path: midterm
      - id: deployment
        uses: actions/deploy-pages@v4
```